

An introduction to interfacing to OpticStudio from Mathematica

or

How to build and optimize a singlet using ZOS-API with Mathematica

written by David Keith

This article

This notebook is intended as an introduction to using ZOS-API to interface to OpticStudio from Mathematica. It is written as a Mathematica version of another knowledgebase article titled: "How to build and optimize a singlet using ZOS-API with Python," written by Thomas Aumeyr. (Thank you, Thomas!) It follows that work step by step, and in fact copies much of the text explanations directly. This has been done so that an immediate comparison can be made between the ZOS-API syntax of Mathematica and Python.

As in that article, we will design a singlet lens with the following specifications:

- Light comes from infinity, with a 5 degree semi-field of view, and a single wavelength (d-line, .587 mm)
- Collimated input light is to be focused to smallest RMS spot size, averaged across the field of view
- F/10, EPD 40 mm
- Made from N-BK7
- Stop is a separate surface and is free to move but comes after the lens
- Lens should be at least 3, and no more than 15 mm thick at center
- Lens should have a minimum edge thickness of 3 mm, and air gaps should be a minimum of 0.5 mm

Revision history

- 1_0 10 July 2019: release.
- 1_0 1/30/2022: works with MMA 13.0 and ZOS 22.1
- 1_1 1/30/2022: Determine Zemax data directory by accessing Registry, works with MMA 13.0 and ZOS 22.1
- 1_1 6/22/2022: Works with ZOS 22.2

.NET Syntax in Mathematica

The Wolfram Language syntax for calling .NET methods and accessing fields is very similar to the syntax used in C# and Visual Basic.NET. The following box compares the Wolfram Language and C# ways of calling constructors, methods, properties, fields, static methods, static properties, and static fields. You can see that Wolfram Language programs that use .NET are written in almost exactly the same way as C# (or VB .NET) programs, except the Wolfram Language uses [] instead of () for arguments and @ instead of the . (dot) as the "member access" operator.

An exception is that for static methods the Wolfram Language uses the context mark ` in place of the dot used by C# and VB. This also parallels the usage in those languages, as their use of the dot in this circumstance is really as a scope resolution operator (like :: in C++). Although the Wolfram Language does not use this terminology, its scope resolution operator is the context mark. .NET namespace names map directly to the Wolfram Language's hierarchical contexts.

Also note that because _ (underscore) is a reserved character in Mathematica, NETLink replaces underscores with a capital U.

Constructors	
C#:	<code>MyClass obj = new MyClass (args) ;</code>
Wolfram Language:	<code>obj = NETNew["MyClass", args] ;</code>
Methods	
C#:	<code>obj.MethodName (args) ;</code>
Wolfram Language:	<code>obj@MethodName [args]</code>
Properties and fields	
C#:	<code>obj.PropertyOrFieldName = 1; value = obj.PropertyOrFieldName;</code>
Wolfram Language:	<code>obj@PropertyOrFieldName = 1; value = obj@PropertyOrFieldName;</code>
Static methods	
C#:	<code>MyClass.StaticMethod (args) ;</code>
Wolfram Language:	<code>MyClass`StaticMethod [args] ;</code>
Static properties and fields	
C#:	<code>MyClass.StaticPropertyOrField = 1; value = MyClass.StaticPropertyOrField;</code>
Wolfram Language:	<code>MyClass`StaticPropertyOrField = 1; value = MyClass`StaticPropertyOrField;</code>

Further information on using .NET/Link, which is the Mathematica .NET interface, can be found at

<http://reference.wolfram.com/language/NETLink/tutorial/Overview.html> or through the help system of a running Mathematica notebook.

Note : Most code lines in this notebook are terminated with a semicolon, which tells Mathematica not to display the output. If you wish to see the output from a particular line - especially if you think something is not working - just delete the semicolon.

Install NetLink

We first need to install Mathematica's NETLink context.

```
In[ ]:= (* loads a Mathematica package *)
Needs["NETLink`"]

In[ ]:= (* calls a function from that package *)
InstallNET[];
```

Determine Zemax data directory

```
In[ ]:= (* Load Registry assembly and type *)
LoadNETAssembly["C:\\Program
Files\\dotnet\\shared\\Microsoft.NETCore.App\\2.1.11\\Microsoft.Win32.Registry.dll"]

Out[ ]:= NETAssembly[Microsoft.Win32.Registry, 1]

In[ ]:= LoadNETType["Microsoft.Win32.Registry"]

Out[ ]:= NETType[Microsoft.Win32.Registry, 1]

In[ ]:= (* Read the registry key value *)
zemaxDataPath = Registry`GetValue["HKEY_CURRENT_USER\\Software\\Zemax", "ZemaxRoot", -1]

Out[ ]:= C:\\Users\\David\\Documents\\Zemax
```

Set ZOS-API directory paths

```
In[ ]:= (* path to the ZOS installation *)
OSPPath = "c:\\program files\\Zemax OpticStudio\\";

In[ ]:= (* path to the ZOS-API Libraries directory *)
librariesPath = zemaxDataPath <> "\\ZOS-API\\Libraries\\"

Out[ ]:= C:\\Users\\David\\Documents\\Zemax\\ZOS-API\\Libraries\\
```

Load Assemblies

Following the Zemax documentation on ZOS-API we load the three ZOS-API assemblies using the *Mathematica* function `LoadNETAssembly`.

```
In[ ]:= helper = LoadNETAssembly[librariesPath <> "\\ZOSAPI_NetHelper.dll"];
In[ ]:= zosapi = LoadNETAssembly[OSPath <> "ZOSAPI.dll"];
In[ ]:= interfaces = LoadNETAssembly[OSPath <> "ZOSAPI_Interfaces.dll"];
```

A quick aside -- Use `NETTypeInfo` to get information

Here is something very useful: *Mathematica .NET/Link* has a function called `NETTypeInfo` which will allow you to quickly display information about the methods, properties, fields, and so on that exist in a given .NET type. You can use this function by passing it each new type, as you go, on down the structure. The ZOS-API fields, properties, methods, etc, are named very logically. You can usually figure out what you can do with an object by passing it to `NETTypeInfo` and reading the results.

For example

```
In[ ]:= NETTypeInfo[zosapi]
Assembly: ZOSAPI
Full Name: ZOSAPI, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
Location: c:\program files\Zemax OpticStudio\ZOSAPI.dll
```

● *Classes*

```
class ZOSAPI.APINetCallback
class ZOSAPI.ZOSAPI_Connection
```

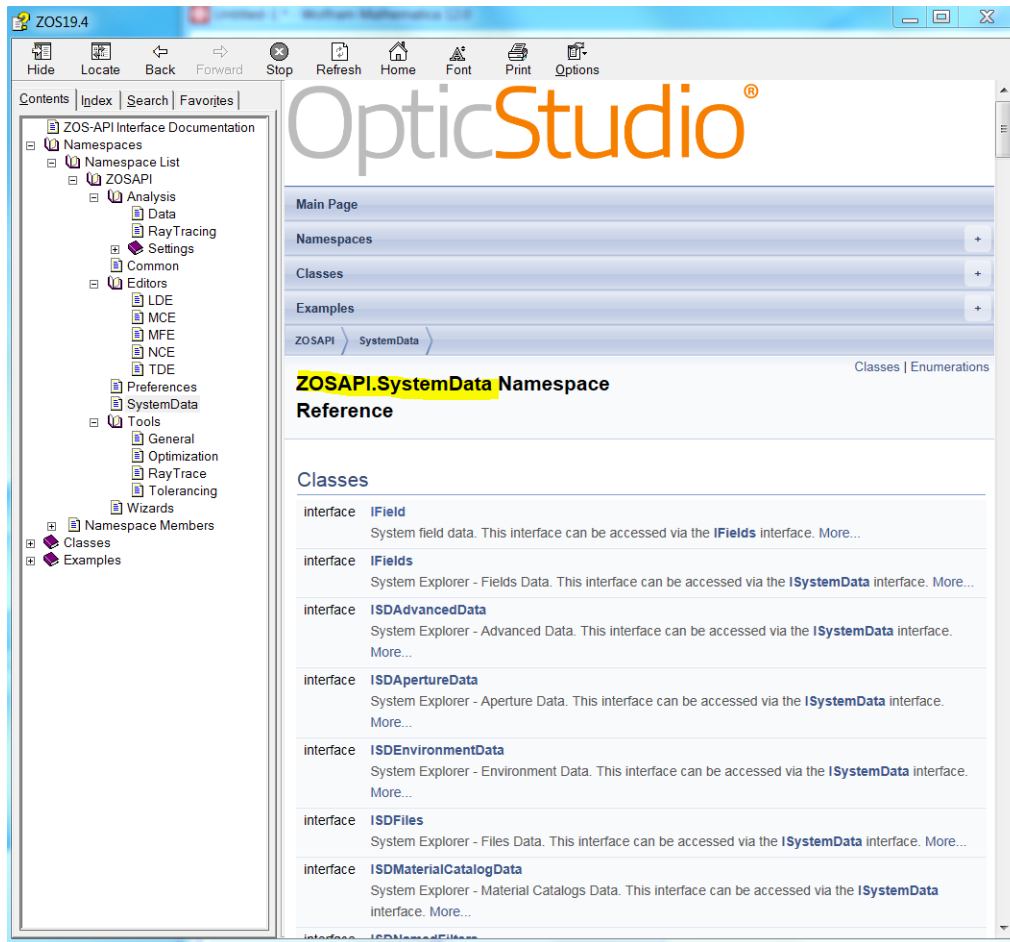
● *Interfaces*

```
interface ZOSAPI.IZOSAPI_Events
```

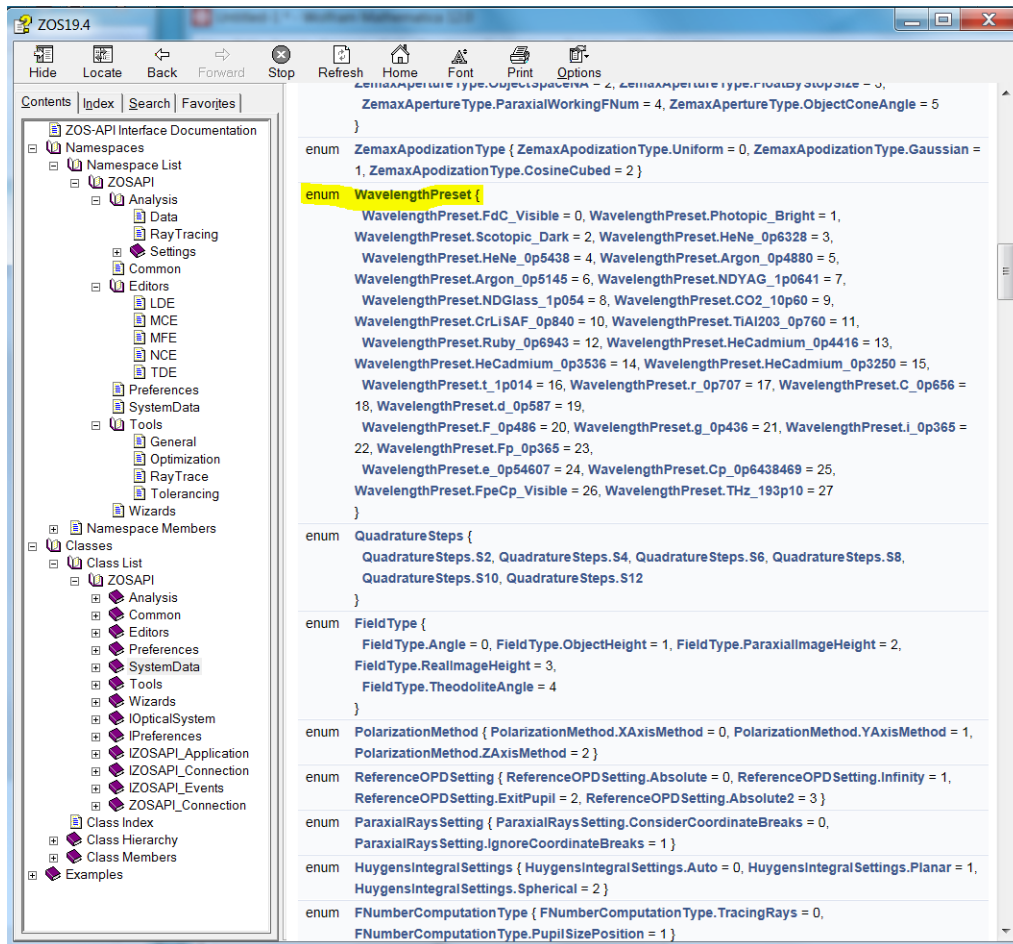
Using enumerated constants

In the work below, we will need to make use of a `WavelengthPreset` which is an enumerated constant. To use the presets we need to load the enumeration and then access the constants by the syntax *Mathematica* uses for accessing contexts.

We can find the needed information in the ZOS-API help system. The wavelength presets are in the system data for OpticStudio. Under the help system, we locate the `SystemData` Namespace. The full name of the namespace is given at the top of the entry as “`ZOSAPI.SystemData`.”



If we scan down to the enumerations section we find the WavelengthPreset declaration.



From this we can see that the full path is “ZOSAPI.SystemData.WavelengthPreset.” We can use that to load the enumeration. This is passed to LoadNETType as a string in quotes.

Notice also that we can see the names of the various enumerated constants in the Help entry. These will be used as symbols, not as strings, so we need to observe the special translation of underscore to U when using them. We can also see the names of the enumerated constants by using NETTypeInfo, as seen below.

Load the NETType

```
In[ ]:= wavelengthPresets = LoadNETType["ZOSAPI.SystemData.WavelengthPreset"]
```

```
Out[ ]:= NETType[ZOSAPI.SystemData.WavelengthPreset, 2]
```

Use NETTypeInfo view the enumerated constants as fields

```
In[ ]:= NETTypeInfo[wavelengthPresets]
```

• Type

```
enum ZOSAPI.SystemData.WavelengthPreset
```

Inheritance:

System.Object

System.ValueType

System.Enum

ZOSAPI.SystemData.WavelengthPreset

Interfaces Implemented: IComparable, IFormattable, IConvertible

Assembly-Qualified Name: ZOSAPI.SystemData.WavelengthPreset, ZOSAPI_Interfaces, Version=1.0

Assembly Location: c:\program files\Zemax OpticStudio\ZOSAPI_Interfaces.dll

• Fields

```
const ZOSAPI.SystemData.WavelengthPreset Argon_0 p4880
const ZOSAPI.SystemData.WavelengthPreset Argon_0 p5145
const ZOSAPI.SystemData.WavelengthPreset C_0 p656
const ZOSAPI.SystemData.WavelengthPreset CO2_10 p60
const ZOSAPI.SystemData.WavelengthPreset Cp_0 p6438469
const ZOSAPI.SystemData.WavelengthPreset CrLiSAF_0 p840
const ZOSAPI.SystemData.WavelengthPreset d_0 p587
const ZOSAPI.SystemData.WavelengthPreset e_0 p54607
const ZOSAPI.SystemData.WavelengthPreset F_0 p486
const ZOSAPI.SystemData.WavelengthPreset FdC_Visible
const ZOSAPI.SystemData.WavelengthPreset Fp_0 p365
const ZOSAPI.SystemData.WavelengthPreset FpeCp_Visible
const ZOSAPI.SystemData.WavelengthPreset g_0 p436
const ZOSAPI.SystemData.WavelengthPreset HeCadmiuim_0 p3250
const ZOSAPI.SystemData.WavelengthPreset HeCadmiuim_0 p3536
const ZOSAPI.SystemData.WavelengthPreset HeCadmiuim_0 p4416
const ZOSAPI.SystemData.WavelengthPreset HeNe_0 p5438
const ZOSAPI.SystemData.WavelengthPreset HeNe_0 p6328
const ZOSAPI.SystemData.WavelengthPreset i_0 p365
const ZOSAPI.SystemData.WavelengthPreset NDGlass_1 p054
const ZOSAPI.SystemData.WavelengthPreset NDYAG_1 p0641
const ZOSAPI.SystemData.WavelengthPreset Photopic_Bright
const ZOSAPI.SystemData.WavelengthPreset r_0 p707
const ZOSAPI.SystemData.WavelengthPreset Ruby_0 p6943
const ZOSAPI.SystemData.WavelengthPreset Scotopic_Dark
const ZOSAPI.SystemData.WavelengthPreset t_1 p014
const ZOSAPI.SystemData.WavelengthPreset THz_193 p10
const ZOSAPI.SystemData.WavelengthPreset TiAl203_0 p760
```

• Methods

int **CompareTo** (object target)

virtual bool **Equals** (object obj)

static bool **Equals** (object objA, object objB)

static string **Format** (Type enumType, object value, string format)

```

virtual int GetHashCode ()
static string GetName (Type enumType, object value)
static string[] GetNames (Type enumType)
Type GetType ()
TypeCode GetTypeCode ()
static Type GetUnderlyingType (Type enumType)
static Array GetValues (Type enumType)
bool HasFlag (Enum flag)
static bool IsDefined (Type enumType, object value)
static object Parse (Type enumType, string value)
static object Parse (Type enumType, string value, bool ignoreCase)
static bool ReferenceEquals (object objA, object objB)
static object ToObject (Type enumType, object value)
static object ToObject (Type enumType, sbyte value)
static object ToObject (Type enumType, short value)
static object ToObject (Type enumType, int value)
static object ToObject (Type enumType, byte value)
static object ToObject (Type enumType, unsigned short value)
static object ToObject (Type enumType, unsigned value)
static object ToObject (Type enumType, long value)
static object ToObject (Type enumType, unsigned long value)
virtual string ToString ()
string ToString (string format, IFormatProvider provider)
string ToString (IFormatProvider provider)
string ToString (string format)
static bool TryParse (string value, out TEnum result)
static bool TryParse (string value, bool ignoreCase, out TEnum result)

```

We can make use of one of the constants using the Mathematica context syntax

Notice the use of the context mark, and that the “_” in the name must be replaced with “U.”

```
In[ ]:= WavelengthPreset`dU0p587
```

```
Out[ ]:= « NETObject[ZOSAPI.SystemData.WavelengthPreset] »
```

Connect and create an application and a primary system

In this section we initialize the interface, make the connection to it from *Mathematica*, and create an application, which is an instance of OpticStudio.

Initialize

ZOSAPI_NetHelper is a static class. We need to load the ZOSAPI_Initializer method and call it as a static

method.

```
In[ ]:= LoadNETType["ZOSAPI_NetHelper.ZOSAPI_Initializer"];
In[ ]:= ZOSAPIUIInitializer`Initialize[];
```

Make a connection and application

```
In[ ]:= theConnection = NETNew["ZOSAPI.ZOSAPI_Connection"];
In[ ]:= theApplication = theConnection@CreateNewApplication[];
```

Check the license status

Mathematica retrieves the enum license type as a .NET object. These can be converted to enum Integers or to a string.

Examine license status

```
In[ ]:= licenseStatus = theApplication@LicenseStatus;
In[ ]:= licenseString = licenseStatus@ToString[]
Out[ ]:= PremiumEdition
```

We can directly check if the license is valid for ZOS-API

```
In[ ]:= (* abort if no valid license *)
If[! theApplication@IsValidLicenseForAPI[],
  MessageDialog["No valid OpticStudio license found"]; Abort[]
]
```

Create a Primary System and save it as a lens file

```
In[ ]:= theSystem = theApplication@PrimarySystem[];
```

The System represents a complete optical system, which corresponds to a single .ZMX file. Now create a string containing the file path for the new file. To make the script flexible, we will use the SamplesDir property to create file path .

```
In[ ]:= (* the name of the lens we are creating *)
fileOut = theApplication@SamplesDir <>
  "\\Sequential\\Objectives\\MMA Single Lens Example wizard+EFFL.zmx";
In[ ]:= (* save the lens *)
theSystem@SaveAs[fileOut]
```

System data

TheSystemData holds all the basic data of the System. We can use this to set aperture, field points, wavelengths, etc.

```
In[ ]:= theSystemData = theSystem@SystemData[ ];

In[ ]:= (* aperture *)
theSystemData@Aperture@ApertureValue = 40;

In[ ]:= (* fields *)
theSystemData@Fields@AddField[0, 5.0, 1.0];

In[ ]:= (* wavelength preset *)

In[ ]:= theSystemData@Wavelengths@SelectWavelengthPreset[WavelengthPreset`dU0p587] ;
```

Create the LDE

The LDE is the Lens Data Editor. We will create and use an instance of the LDE to modify the lens as needed.

```
In[ ]:= theLDE = theSystem@LDE

Out[ ]:= « NETObject[ZemaxUI.ZOSAPI.Editors.ZOSAPI_LDE] »
```

And use it to set up the lens

In the LDE, each row represents a surface defined in the System: Surface 0 is the Object and the last Surface is the Image. For a simple lens located before the stop, the front side of the lens would be Surface 1, the rear side of the lens is Surface 2 and the Stop would be at Surface 3.

Lens data

```
In[ ]:= theLDE@InsertNewSurfaceAt[1];

In[ ]:= theLDE@InsertNewSurfaceAt[1];

In[ ]:= surface1 = theLDE@GetSurfaceAt[1];

In[ ]:= surface2 = theLDE@GetSurfaceAt[2];

In[ ]:= surface3 = theLDE@GetSurfaceAt[3];

In[ ]:= surface1@Thickness = 10.0;

In[ ]:= surface1@Comment = "Front of lens";

In[ ]:= surface1@Material = "N-BK7";
```

```

In[ ]:= surface2@Thickness = 50.0;

In[ ]:= surface2@Comment = "Rear of lens";

In[ ]:= surface3@Comment = "Stop is free to move";

In[ ]:= surface3@Thickness = 350.0;

```

Make thicknesses and radii variable

We have 5 independent variables, which we can use to achieve our goal – the front and back radius of the lens, the lens thickness, the location of the stop and the image location.

```

In[ ]:= surface1@RadiusCell@MakeSolveVariable[];

In[ ]:= surface1@ThicknessCell@MakeSolveVariable[];

In[ ]:= surface2@RadiusCell@MakeSolveVariable[];

In[ ]:= surface2@ThicknessCell@MakeSolveVariable[];

In[ ]:= surface3@ThicknessCell@MakeSolveVariable[];

```

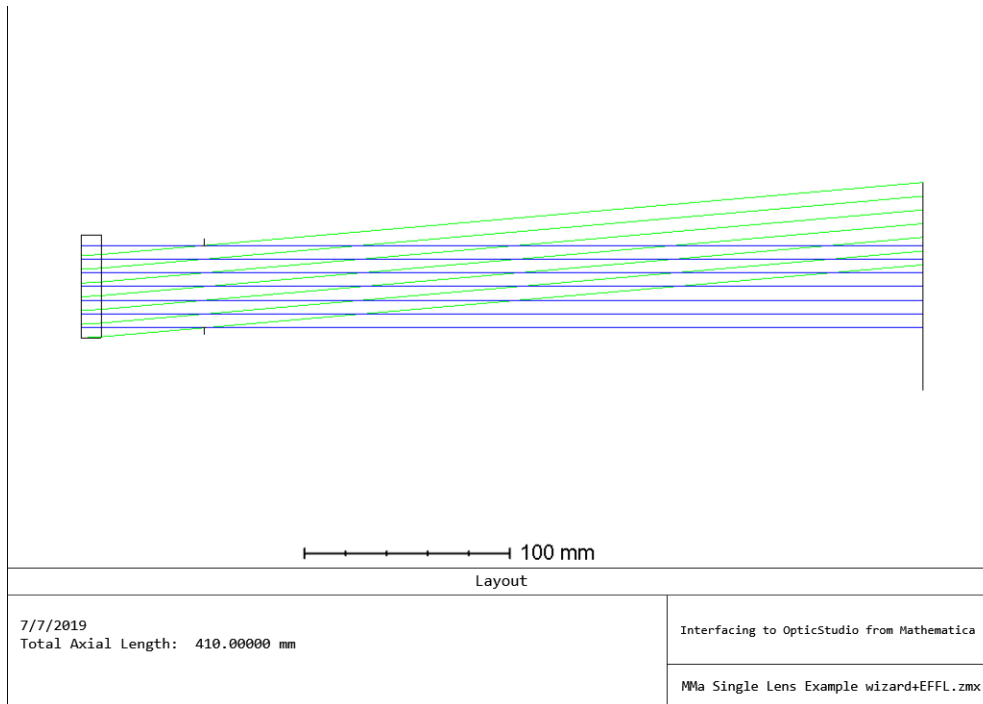
After this, the LDE will look like this :

```

In[ ]:= (* save the lens *)
theSystem@Save[]

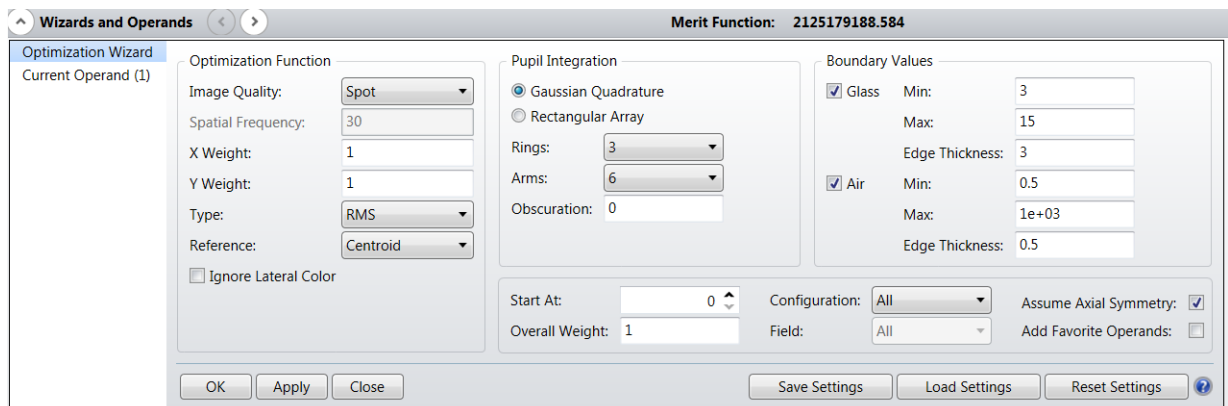
```

	Surface Type	Comment	Radius	Thickness	Material	Coating	Clear Semi-Dia	Chip Zone	Mech Semi-Dia	Conic	TCE x 1E-6
0	OBJECT Standard ▾		Infinity	Infinity			Infinity	0.000	Infinity	0.000	0.000
1	Standard ▾	Front of lens	Infinity V	10.000 V	N-BK7		24.951	0.000	24.951	0.000	-
2	Standard ▾	Rear of lens	Infinity V	50.000 V			24.376	0.000	24.951	0.000	0.000
3	STOP Standard ▾	Stop is free to move	Infinity	350.000 V			20.001	0.000	20.001	0.000	0.000
4	IMAGE Standard ▾		Infinity	-			50.620	0.000	50.620	0.000	0.000



Setting up the Merit Function Editor

We will use the sequential Optimization Wizard to make a Default Merit Function target the Minimum Spot Radius also specifying the Glass/Air Boundary Values.



Create and populate the merit function editor

```

In[ ]:= theMFE = theSystem@MFE;

In[ ]:= wizard = theMFE@SEQOptimizationWizard;

In[ ]:= wizard@Type = 0; (*RMS*)

In[ ]:= wizard@Data = 1; (*Spot Radius*)

```

```

In[ ]:= wizard@Reference = 0; (*Centroid*)

In[ ]:= wizard@Ring = 2; (*3 Rings*)

In[ ]:= wizard@Arm = 6; (*6 Arms *)

In[ ]:= wizard@IsGlassUsed = True;

In[ ]:= wizard@GlassMin = 3;

In[ ]:= wizard@GlassMax = 15;

In[ ]:= wizard@GlassEdge = 3;

In[ ]:= wizard@IsAirUsed = True;

In[ ]:= wizard@AirMin = 0.5;

In[ ]:= wizard@AirMax = 1000;

In[ ]:= wizard@AirEdge = 0.5;

In[ ]:= wizard@IsAssumeAxialSymmetryUsed = True;

In[ ]:= wizard@CommonSettings@OK[];

```

The OK button method is part of the CommonSettings of all wizards available in OpticStudio.

Since the lens is before the stop, we cannot use a F/# solve. We don't know which ray is the marginal ray until it has hit the edge of the stop. This means we must use the EFFL operand in addition to the RMS spot default merit function. Just add an extra line at the top, target the EFL to 400mm. Remember to add a weight of 1.

```

In[ ]:= operand1 = theMFE@InsertNewOperandAt[1];

In[ ]:= (* load and use the enumeration *)
meritOperandTypes = LoadNETType["ZOSAPI.Editors.MFE.MeritOperandType"];
operand1@ChangeType[MeritOperandType`EFFL];

In[ ]:= operand1@Target = 400.0;

In[ ]:= operand1@Weight = 1.0;

```

After this, the Merit Function will look like this

```

In[ ]:= (* save the lens *)
theSystem@Save[];

```

Merit Function Editor

Wizards and Operands Merit Function: Click Update

	Type	Wave					Target	Weight	Value	% Contrib
1	EFFL	1					400.000	1.000	0.000	0.000
2	DMFS									
3	BLNK		Sequential merit function: RMS spot radius centroid GQ 3 rings 18 arms							
4	BLNK		Default individual air and glass thickness boundary constraints.							
5	MNCA	1	1				0.500	1.000	0.000	0.000
6	MXCA	1	1				1.000E+03	1.000	0.000	0.000
7	MNEA	1	1 0.000	0			0.500	1.000	0.000	0.000
8	MNCG	1	1				3.000	1.000	0.000	0.000
9	MXCG	1	1				15.000	1.000	0.000	0.000
10	MNEG	1	1 0.000	0			3.000	1.000	0.000	0.000
11	MNCA	2	2				0.500	1.000	0.000	0.000
12	MXCA	2	2				1.000E+03	1.000	0.000	0.000
13	MNEA	2	2 0.000	0			0.500	1.000	0.000	0.000
14	MNCG	2	2				3.000	1.000	0.000	0.000
15	MXCG	2	2				15.000	1.000	0.000	0.000
16	MNEG	2	2 0.000	0			3.000	1.000	0.000	0.000
17	MNCA	3	3				0.500	1.000	0.000	0.000
18	MXCA	3	3				1.000E+03	1.000	0.000	0.000
19	MNEA	3	3 0.000	0			0.500	1.000	0.000	0.000
20	MNCG	3	3				3.000	1.000	0.000	0.000
21	MXCG	3	3				15.000	1.000	0.000	0.000
22	MNEG	3	3 0.000	0			3.000	1.000	0.000	0.000
23	BLNK		Operands for field 1.							
24	TRAC		1 0.000	0.000	0.336	0.000	0.000	0.436	0.000	0.000
25	TRAC		1 0.000	0.000	0.707	0.000	0.000	0.698	0.000	0.000
26	TRAC		1 0.000	0.000	0.942	0.000	0.000	0.436	0.000	0.000
27	BLNK		Operands for field 2.							
28	TRAC		1 0.000	1.000	0.058	0.331	0.000	0.048	0.000	0.000
29	TRAC		1 0.000	1.000	0.123	0.696	0.000	0.078	0.000	0.000
30	TRAC		1 0.000	1.000	0.164	0.928	0.000	0.048	0.000	0.000
31	TRAC		1 0.000	1.000	0.168	0.291	0.000	0.048	0.000	0.000
32	TRAC		1 0.000	1.000	0.354	0.612	0.000	0.078	0.000	0.000
33	TRAC		1 0.000	1.000	0.471	0.816	0.000	0.048	0.000	0.000
34	TRAC		1 0.000	1.000	0.257	0.316	0.000	0.048	0.000	0.000

Local Optimization

Now let's access the properties and methods for running a Local Optimization and set it up like in the Graphical User Interface.

Local Optimization

Algorithm: Damped Least Squares # of Cores: 8

Targets: 49 Cycles: Automatic

Variables: 5 Status: Idle

Initial Merit Function: 2.1252e+09 Execution Time:

Current Merit Function: 2.1252e+09

☐ Auto Update Start Stop Exit Save Load Reset ?

Set up the optimizer

```
In[ ]:= localOpt = theSystem@Tools@OpenLocalOptimization[];
```

```
In[ ]:= LoadNETType["ZOSAPI.Tools.Optimization.OptimizationAlgorithm"];
```

```
In[ ]:= localOpt@Algorithm = OptimizationAlgorithm`DampedLeastSquares;
```

```

In[ ]:= LoadNETType["ZOSAPI.Tools.Optimization.OptimizationCycles"];

In[ ]:= localOpt@Cycles = OptimizationCycles`Automatic;

In[ ]:= localOpt@NumberOfCores = 8;

```

Run the optimization

```

In[ ]:= localOpt@RunAndWaitForCompletion[];

```

And close the tool

```

In[ ]:= localOpt@Close[];

```

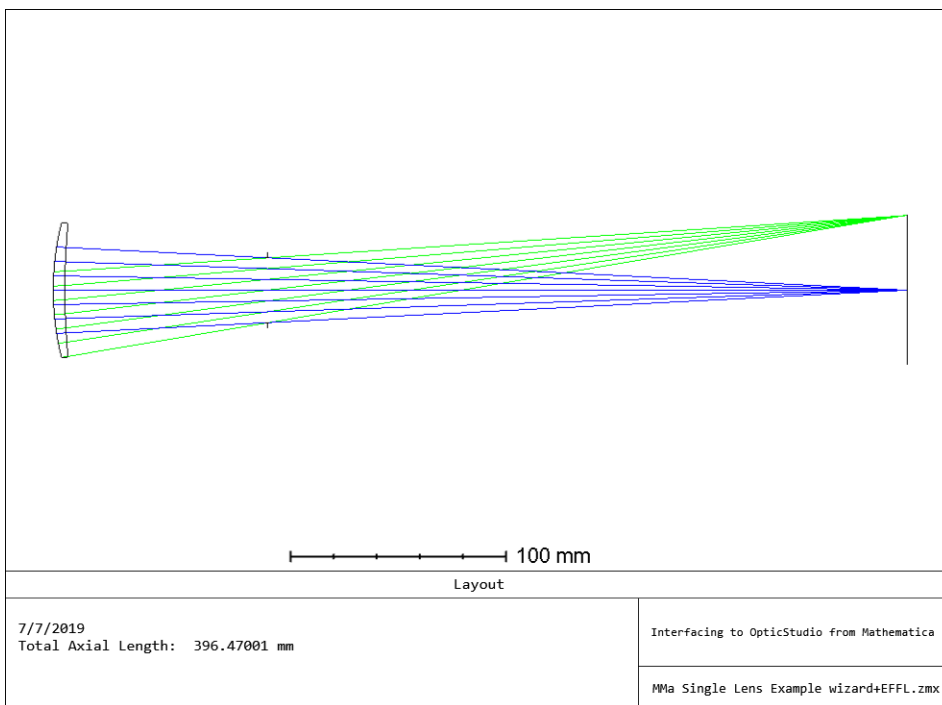
The result

After running your script, open OpticStudio and check out your fully optimised lens.

```

In[ ]:= (* save the lens *)
theSystem@Save[];

```



Close the application

Uncomment this line if you want to close the application.

```

In[ ]:= (*theApplication@CloseApplication[] *)

```